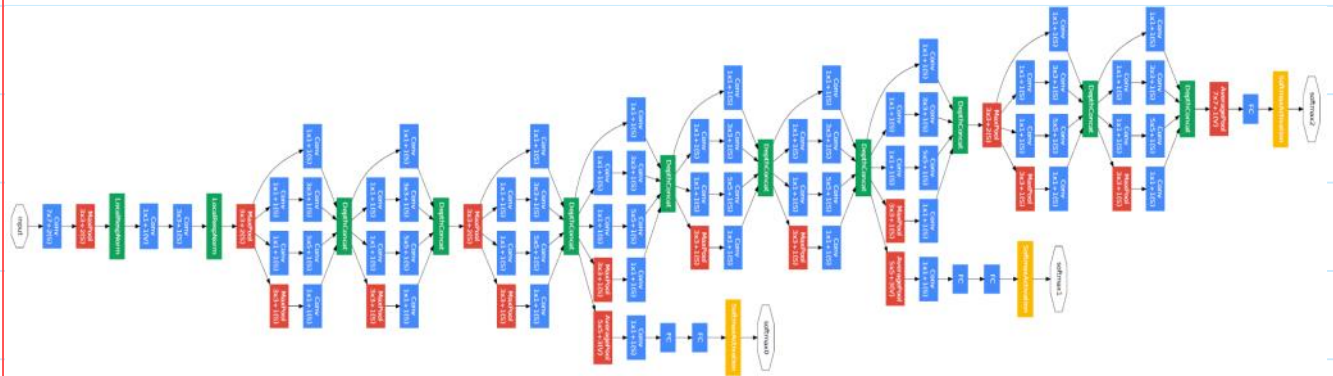
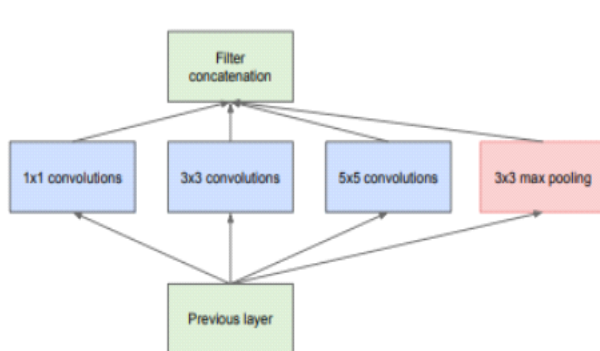


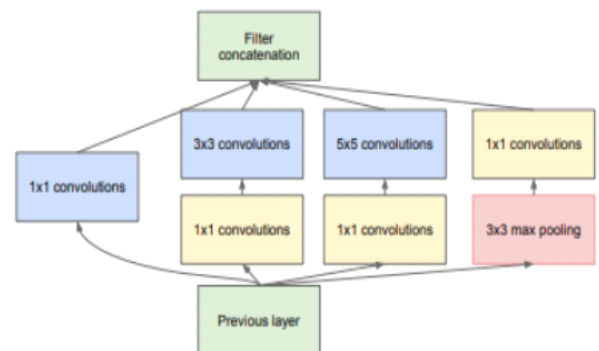
GoogleNet



- ImageNet Challenge 2014 우승 모델(* VGG는 2014 2등)
- 네트워크가 깊어지면 발생하는 문제점(Overfitting , 파라미터 수 증가)



(a) Inception module, naïve version



(b) Inception module with dimension reductions

Figure 2: Inception module

```

import torch
import torch.nn as nn
from torchsummary import summary

class InceptionModule_naive(nn.Module):
    def __init__(self, in_channels):
        super(InceptionModule_naive, self).__init__()

        self.branch1 = nn.Conv2d(in_channels, 64, kernel_size=1)

        self.branch2 = nn.Conv2d(in_channels, 128, kernel_size=3, padding=1)

        self.branch3 = nn.Conv2d(in_channels, 32, kernel_size=5, padding=2)

        self.branch4 = nn.MaxPool2d(kernel_size=3, stride=1, padding=1)

    def forward(self, x):
        branch1_out = self.branch1(x)
        branch2_out = self.branch2(x)
        branch3_out = self.branch3(x)
        branch4_out = self.branch4(x)

        outputs = torch.cat([branch1_out, branch2_out, branch3_out, branch4_out], dim=1)
        return outputs

x = torch.randn(1, 192, 224, 224)
model = InceptionModule_naive(in_channels=192)
output = model(x)
print("Output shape:", output.shape)

summary(model, input_size=(192, 224, 224))

```

```

=====
Layer (type)           Output Shape          Param #
=====
          Conv2d-1      [-1, 64, 224, 224]    12,352
          Conv2d-2      [-1, 128, 224, 224]   221,312
          Conv2d-3      [-1, 32, 224, 224]    153,632
        MaxPool2d-4     [-1, 192, 224, 224]      0
=====
Total params: 387,296
Trainable params: 387,296
Non-trainable params: 0
=====
Input size (MB): 36.75
Forward/backward pass size (MB): 159.25
Params size (MB): 1.48
Estimated Total Size (MB): 197.48

```

```

import torch
import torch.nn as nn
from torchsummary import summary

class InceptionModule_with_reduction(nn.Module):
    def __init__(self, in_channels):
        super(InceptionModule_with_reduction, self).__init__()

        self.branch1 = nn.Conv2d(in_channels, 64, kernel_size=1)

        self.branch2_1 = nn.Conv2d(in_channels, 96, kernel_size=1)
        self.branch2_2 = nn.Conv2d(96, 128, kernel_size=3, padding=1)

        self.branch3_1 = nn.Conv2d(in_channels, 16, kernel_size=1)
        self.branch3_2 = nn.Conv2d(16, 32, kernel_size=5, padding=2)

        self.branch4_pool = nn.MaxPool2d(kernel_size=3, stride=1, padding=1)
        self.branch4_conv = nn.Conv2d(in_channels, 32, kernel_size=1)

    def forward(self, x):
        branch1_out = self.branch1(x)

        branch2_out = self.branch2_2(self.branch2_1(x))

        branch3_out = self.branch3_2(self.branch3_1(x))

        branch4_out = self.branch4_conv(self.branch4_pool(x))

        outputs = torch.cat([branch1_out, branch2_out, branch3_out, branch4_out], dim=1)
        return outputs

x = torch.rand(1, 192, 224, 224)
model = InceptionModule_with_reduction(in_channels=192)
output = model(x)
print("Output shape:", output.shape)

summary(model, input_size=(192, 224, 224))

```

Layer (type)	Output Shape	Param #
Conv2d-1	[-1, 64, 224, 224]	12,352
Conv2d-2	[-1, 96, 224, 224]	18,528
Conv2d-3	[-1, 128, 224, 224]	110,720
Conv2d-4	[-1, 16, 224, 224]	3,088
Conv2d-5	[-1, 32, 224, 224]	12,832
MaxPool2d-6	[-1, 192, 224, 224]	0
Conv2d-7	[-1, 32, 224, 224]	6,176
=====		
Total params: 163,696		
Trainable params: 163,696		
Non-trainable params: 0		
=====		
Input size (MB): 36.75		
Forward/backward pass size (MB): 214.38		
Params size (MB): 0.62		
Estimated Total Size (MB): 251.75		
=====		

Layer (type)	Output Shape	Param #
Conv2d-1	[-1, 64, 224, 224]	12,352
Conv2d-2	[-1, 128, 224, 224]	221,312
Conv2d-3	[-1, 32, 224, 224]	153,632
MaxPool2d-4	[-1, 192, 224, 224]	0
Total params: 387,296		
Trainable params: 387,296		
Non-trainable params: 0		
Input size (MB): 36.75		
Forward/backward pass size (MB): 159.25		
Params size (MB): 1.48		
Estimated Total Size (MB): 197.48		

Layer (type)	Output Shape	Param #
Conv2d-1	[-1, 64, 224, 224]	12,352
Conv2d-2	[-1, 96, 224, 224]	18,528
Conv2d-3	[-1, 128, 224, 224]	110,720
Conv2d-4	[-1, 16, 224, 224]	3,088
Conv2d-5	[-1, 32, 224, 224]	12,832
MaxPool2d-6	[-1, 192, 224, 224]	0
Conv2d-7	[-1, 32, 224, 224]	6,176
Total params: 163,696		
Trainable params: 163,696		
Non-trainable params: 0		
Input size (MB): 36.75		
Forward/backward pass size (MB): 214.38		
Params size (MB): 0.62		
Estimated Total Size (MB): 251.75		

DenseNetNet

- 2017년 발표, Densely Connected Convolutional Networks
- ResNet과 비교하여 더 적은 파라미터로 유사한 성능
- "특징의 재활용", "정보 흐름 강화"에 초점

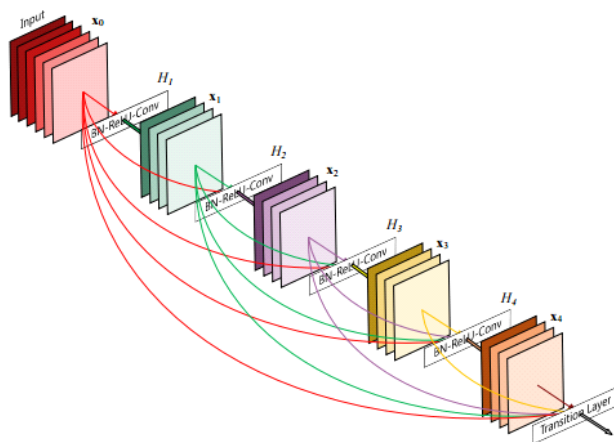
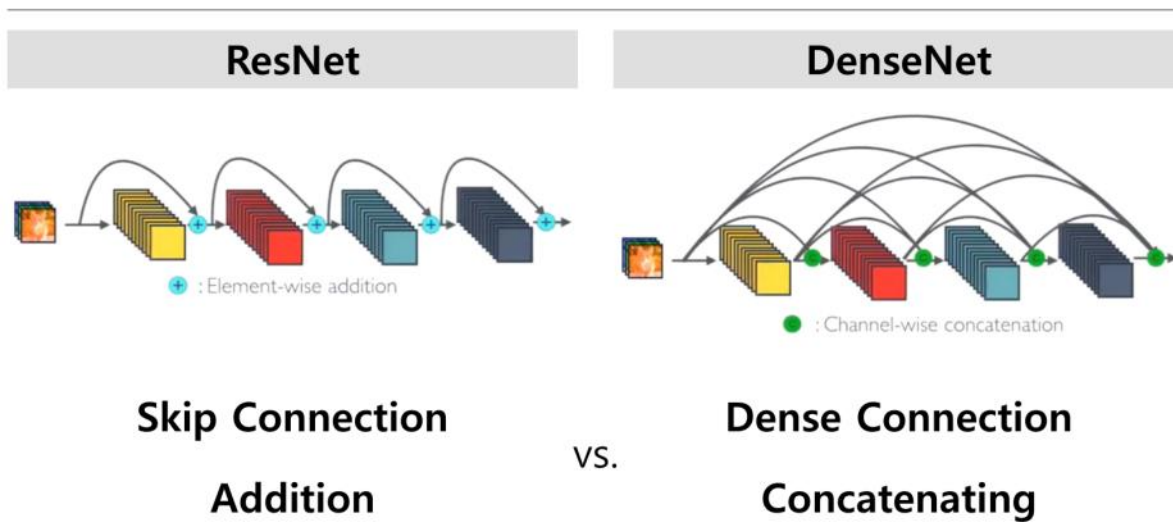
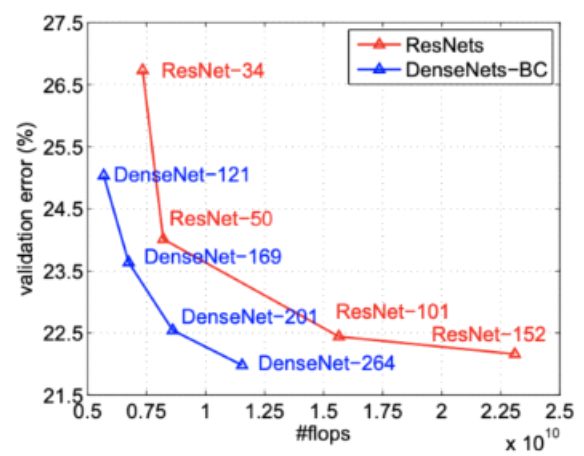
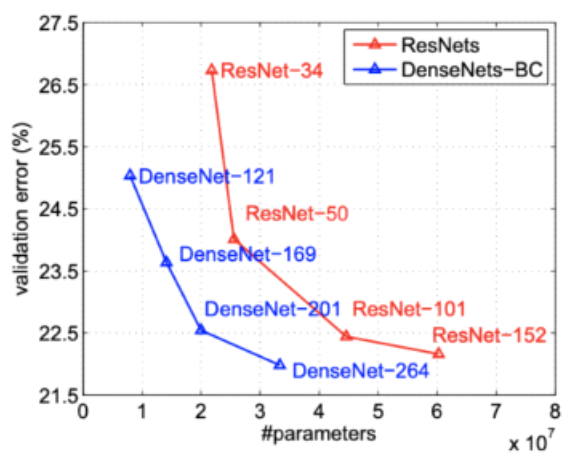
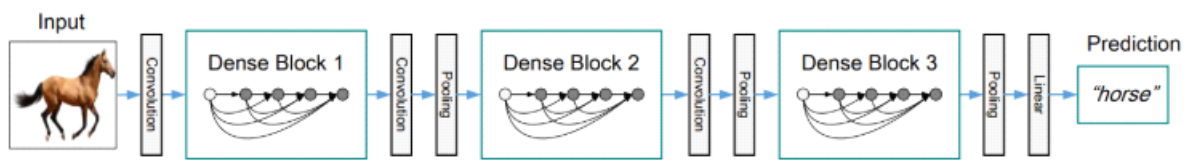


Figure 1: A 5-layer dense block with a growth rate of $k = 4$. Each layer takes all preceding feature-maps as input.

ResNet vs. DenseNet





과제 내용 : Googlenet의 auxiliary classifier는 중간 layer의 결과를 최종 loss에 합산하여 loss를 증가시켜 결과적으로 gradient를 크게 만든다. 최신 딥러닝 모델에서는 이러한 아이디어(방법)이 거의 사용되지 않는데, 왜 사용하지 않을까에 대해 본인의 생각을 작성하여 제출하시오.

과제 점수 : 2점

과제 제출 : 자유 형식으로 작성후 pdf 변환하여 제출(1장 이내)

과제 제출 형식 : pdf (형식 미준수시 감점 -0.5)

과제 제출 기한 : 5월 20일(화) 23시59분

*** ChatGPT 사용 불가, ChatGPT 사용하여 작성 금지, 잘못된 결과라도 상관없으니 본인이 위의 이유에 대해 생각해보고, 본인 생각을 "왜??" 라는 관점에서 설명.